

Microcontrôleur

LEVY Noam 371921

FOUET Axel 375825

mai 2025

1 Introduction

Ce projet consiste en l'implémentation du jeu **Tetris** sur une plateforme à microcontrôleur. L'application utilise une matrice LED 8x8 pour l'affichage, un encodeur angulaire pour le contrôle utilisateur, un écran LCD pour afficher des messages et informations, ainsi qu'un système d'interruptions pour la mise en pause.

L'objectif principal est de concevoir un système interactif et réactif capable de faire fonctionner une version rudimentaire mais fonctionnelle de Tetris avec des contraintes temps réel.

Rappel des critères :

- utiliser au minimum trois périphériques (dont le panneau LED WS2812B)
- utiliser une ou plusieurs interruptions
- mesurer et traiter une quantité ou valeur, ou traiter une information externe, et communiquer un résultat ou contrôler un dispositif
- posséder une interface utilisateur permettant d'entrer des paramètres
- afficher un résultat sur LCD ou terminal par UART (RealTerm) ou autre
- utiliser les macros et bibliothèques présentées au cours

2 Description générale de notre application

Afin de répondre à ces attentes, nous avons décidé de recréer un jeu emblématique, le Tetris. À l'aide du panneau LED, nous avons pu recréer des blocs de quatre pixels agencés de diverses manières, qui chutent vers le bas de l'écran et s'arrêtent lorsqu'ils rencontrent le bas du panneau ou bien un autre bloc.

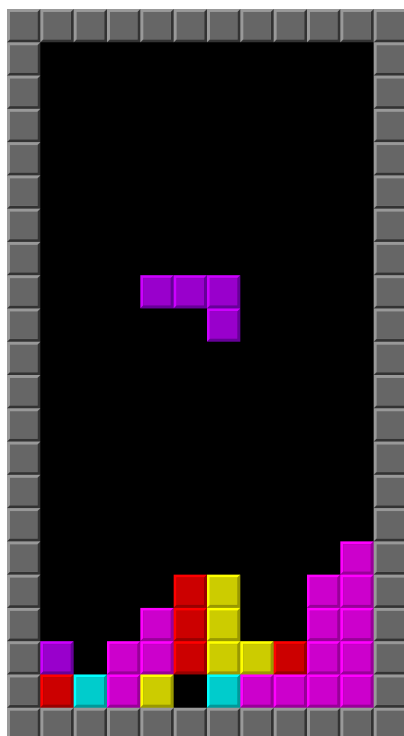
Une fois les blocs dans leur position finale, si une ligne de pixels est formée, elle disparaît, et tous les pixels posés au-dessus descendent d'un cran.

Les blocs sont contrôlables pour aller de gauche à droite à l'aide d'un encodeur angulaire. Lorsqu'on tourne l'encodeur dans le sens des aiguilles d'une montre, le bloc se déplace vers la droite. Lorsqu'on tourne l'encodeur dans le sens inverse des aiguilles d'une montre, le bloc se déplace vers la gauche. L'encodeur angulaire, muni d'un bouton-poussoir, permet également de lancer le Tetris.

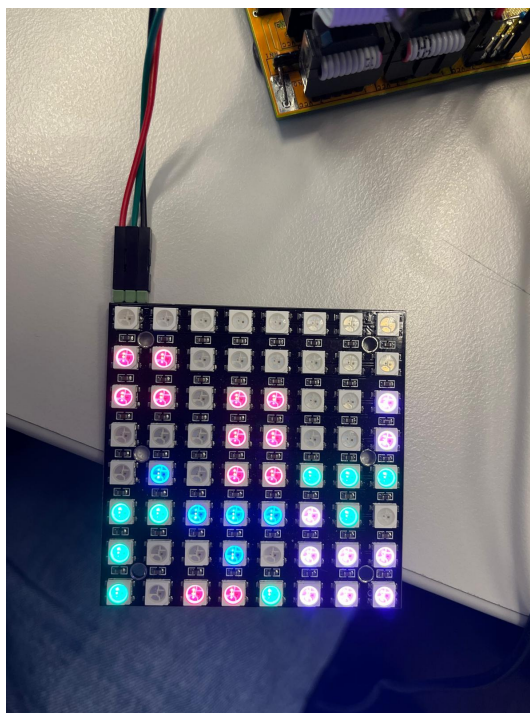
Enfin, avant de commencer la partie, il est possible d'entrer votre pseudo sur l'écran LCD à l'aide des boutons du microcontrôleur. Il sera alors enregistré dans la mémoire EEPROM. À la fin de votre partie, votre score sera également enregistré dans la mémoire EEPROM. Ainsi, à la partie suivante, le nom du joueur précédent et son score seront affichés sur l'écran LCD. À la fin de la partie, votre score s'affiche.

Enfin, il est possible de déclencher une interruption, à l'aide d'un bouton du microcontrôleur, qui met en pause le jeu lors d'une éventuelle distraction extérieure.

Ainsi, avec toutes ces implémentations, nous remplissons toutes les exigences du projet.



(a) Version originale



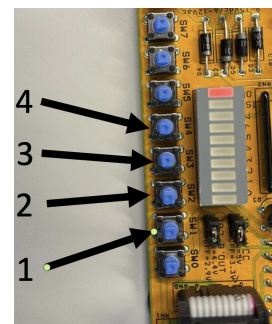
(b) Version de notre projet

Figure 1: Le jeu Tetris

3 Mode d'emploi

Le programme est conçu pour fonctionner à 4 MHz, il faut donc flasher le programme en 4 MHz. Une fois le programme lancé, il vous est possible de rentrer votre pseudo. Pour ce faire, il faut utiliser les boutons 1, 2, 3 et 4. Le bouton 1 vous permet de sélectionner une lettre en faisant défiler l'alphabet dans le sens croissant. Le second bouton vous permet de changer la lettre vers l'arrière. Le bouton 3 sert à sélectionner la prochaine lettre. Et le dernier bouton sert à recommencer la sélection.

Une fois le pseudo rentré, en appuyant sur l'encodeur angulaire, vous pouvez commencer le jeu. Une fois le jeu Tetris commencé, vous pouvez choisir l'emplacement des pièces en tournant l'encodeur angulaire vers la droite ou la gauche. Il vous est possible à tout moment de mettre en pause le jeu à l'aide d'une interruption en appuyant sur le bouton 0. Une fois le jeu perdu, votre score s'affiche durant trois secondes, puis vous êtes remis dans le menu principal.



4 Description technique de l'application et du matériel

4.1 Utilisation des ports

Le système repose sur plusieurs interfaces physiques connectées aux ports du microcontrôleur **ATmega128** :

- **Port E** : utilisé pour lire les signaux de l'*encodeur angulaire*, permettant de détecter les rotations de l'utilisateur.
- **Port B** : la broche PB1 (pin 1 du Port B) est utilisée comme ligne de commande pour le *panneau de 64 LED WS2812B*. Ce composant nécessite une transmission série avec un protocole temporel strict, gérée par des macros car sensible aux délais.
- **Port D** : utilisé pour les *boutons-poussoirs*. Chaque bouton est connecté à une entrée numérique. Le bouton D0, mappé sur l'int0, déclenche une interruption externe pour mettre le jeu en pause.

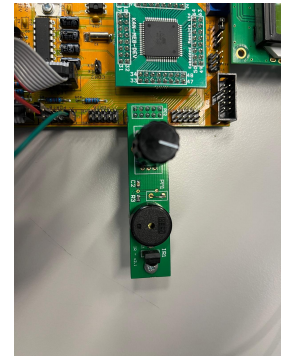
- **Interface LCD** : l'écran LCD est connecté via une interface parallèle, utilisant plusieurs broches numériques (utilise le Port A et une partie du Port C).

4.2 Interfaces d'acquisition et d'affichage

Cette application repose sur plusieurs interfaces pour interagir avec l'utilisateur et afficher les données en temps réel :

Acquisition de données

- **Encodeur angulaire** : permet de naviguer dans les menus et de déplacer les pièces durant la partie. Le sens de rotation est détecté en lecture continue.
- **Boutons** : connectés au PORTD, ils sont utilisés pour la sélection de lettres lors de l'entrée du nom. L'appui sur le bouton 0 déclenche une interruption externe (INT0) pour lancer ou mettre en pause la partie.



Affichage des données

- **Écran LCD** : utilisé pour afficher le nom de l'utilisateur, le score actuel ou précédent. L'affichage est géré via des sous-routines comme `LCD_putc`, `LCD_pos` et `LCD_clear`. Lors du démarrage, le LCD affiche une invite à entrer un nom. Une fois la partie perdue, le nom et le score sont enregistrés dans l'EEPROM, puis affichés à nouveau.
- **Matrice LED 8x8 RGB** : sert d'interface graphique principale pour le jeu Tetris. Chaque pixel est contrôlé via des buffers mémoire représentant le plateau de jeu et la pièce en mouvement. L'affichage est mis à jour régulièrement par la routine `display_screen_and_moving`.

EEPROM Les noms des joueurs et leurs scores sont sauvegardés dans la mémoire EEPROM afin d'être persistants entre deux sessions. Lors du démarrage, le dernier score enregistré est récupéré et affiché. À la fin de la partie, le score du joueur actif est écrit à nouveau dans l'EEPROM.

4.3 Périphériques utilisés

- Encodeur rotatif – Port E
- Écran LCD – Interface parallèle (Port A)
- Panneau de LED WS2812B – Port B, PB1
- Boutons-poussoirs – Port D

4.4 Interruptions

Le système utilise une interruption externe via **INT0** (connecté à PD0) :

- **Déclenchement** : pression du bouton 0.
- **Effet** : met en pause le jeu immédiatement.
- **Traitement** : dans la routine `ISR(INT0_vect)`, un flag de pause est modifié. Ce flag est ensuite lu dans la boucle principale pour geler temporairement le déroulement du programme.

4.5 Fonctionnement du jeu (top-down)

Le programme suit une architecture descendante, organisée en étapes claires dès le démarrage :

1. **reset** : initialise les ports, le stack pointer, les interruptions, l'encodeur et l'affichage LCD.
2. **main** : boucle principale qui gère l'entrée du nom via l'encodeur. Lorsqu'on appuie sur l'encodeur, le jeu démarre.
3. **start_game** : initialise les structures mémoire pour le jeu (pièces, écran, tests de collision).
4. **main_game_loop** : boucle de jeu principale où la pièce est affichée, déplacée et descendue. Les entrées utilisateur sont lues et les collisions sont gérées.
5. **Fin de partie** : si la partie est perdue (collision au chargement d'une nouvelle pièce), le score et le nom du joueur sont sauvegardés dans l'EEPROM, puis le score est affiché sur le LCD.

Chaque étape fait appel à des sous-modules spécialisés pour l'affichage, les mouvements, les tests de collision ou la gestion de la mémoire.

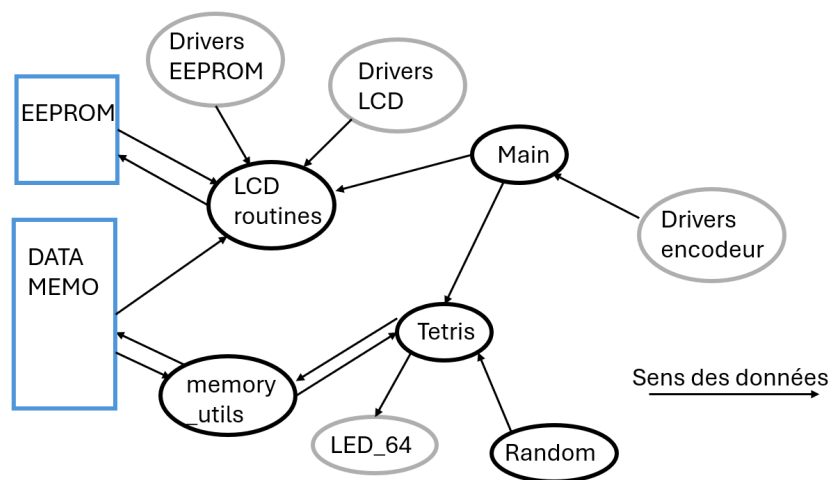


Figure 2: Schéma des dépendances

4.6 Présentation des modules (parties indépendantes du programme)

Le projet est structuré en plusieurs fichiers ASM, chacun remplissant un rôle précis :

- **main.asm** : point d'entrée du programme. Il contient la logique de séquence, l'initialisation générale, la boucle principale et les interruptions.
- **tetris.asm** : contient toute la logique du jeu Tetris : affichage à l'écran, déplacements des pièces, gestion du score, détection de collision, chargement de nouvelles pièces, game over, etc. .
- **lcd_routine.asm** : routines plus haut niveau pour la gestion de l'affichage de messages sur l'écran LCD.
- **angular_encoder.asm** : lecture des signaux de l'encodeur angulaire pour la navigation dans les menus et le contrôle des pièces.
- **memory_utils.asm** : macros d'abstraction pour initialiser, copier, déplacer et manipuler la mémoire (notamment les buffers des pièces et de l'écran).
- **eeeprom.asm** : lecture et écriture dans la mémoire EEPROM pour sauvegarder et restaurer le nom et le score du dernier joueur.
- **random.asm** : génération pseudo-aléatoire pour la sélection des pièces Tetris.

Chaque module est autonome et permet une organisation claire du code, facilitant le débogage et les extensions potentielles.

4.7 Tentative d'utilisation

La première idée de notre projet était d'utiliser un détecteur de distance connecté à un servomoteur afin de créer un distributeur de friandises. Nous avons trouvé une application rare mais pratique du microcontrôleur : l'utilisation du comparateur analogique. Ce comparateur aurait dû comparer la tension analogique envoyée par le détecteur de distance à une tension seuil. Cette tension seuil aurait pu être définie à l'aide d'une tension interne du microcontrôleur ou en utilisant un générateur de tension continue connecté à la broche 3 du port E. La tension envoyée par le détecteur devait être appliquée à la broche 2 du port E. Ainsi, le comparateur, connecté en matériel à ces deux broches, envoie une interruption lorsque la tension analogique du détecteur dépasse la tension seuil. Cette interruption aurait permis de déclencher le servomoteur.

Tous les réglages du comparateur (tension de référence interne ou externe, interruption sur front montant ou descendant, etc.) sont gérés par le registre ACSR. Malheureusement, malgré l'aide précieuse de trois assistants simultanés et compte tenu de la rareté de l'utilisation de cette fonctionnalité, nous n'avons pas réussi à le faire fonctionner.

5 Description de détail de l'accès aux périphériques

5.1 Connexion hardware

Afin d'accéder à un périphérique, la première étape est de le câbler correctement.

- **Périphérique 64 LED WS2812B** : ce périphérique fonctionne à l'aide de trois câbles séparés. Branchez le câble **VCC** et **GND** sur les entrées **VCC** et **GND** du **PORTB**. Puis, branchez le câble **signal** sur le **PIN1** du **PORTB**.
- **Périphérique encodeur angulaire** : ce périphérique est directement connecté à une plaque de silicium à dix entrées. Ainsi, le nombre d'entrées correspond directement aux dix pins du **PORTE**. Le **VCC** et le **GND** sont directement bien placés, et il faudra connecter les câbles de signal aux bons pins à l'aide du software (cf. 5.2).
- **Périphérique LCD** : sur la carte, une zone LCD est directement prévue pour ce périphérique. Le **VCC** et le **GND** sont également placés automatiquement. Une subtilité importante doit être prise en compte lors de l'utilisation du LCD. En effet, en plus du port prévu directement pour ce dernier périphérique, le LCD utilise également les **PORTA** et **PORTC**. Il est donc nécessaire de ne rien connecter d'autre à ces ports sous peine de détruire les informations envoyées par la mémoire dans un autre contexte que le LCD.
- **Périphérique boutons-poussoirs** : la carte est munie d'un port **SWITCH** non accessible par le software. À l'aide des câbles gris qui permettent de connecter dix pins à dix autres pins, on peut connecter le port **SWITCH** au **PORTD**. Ainsi, les informations deviennent accessibles par le software.

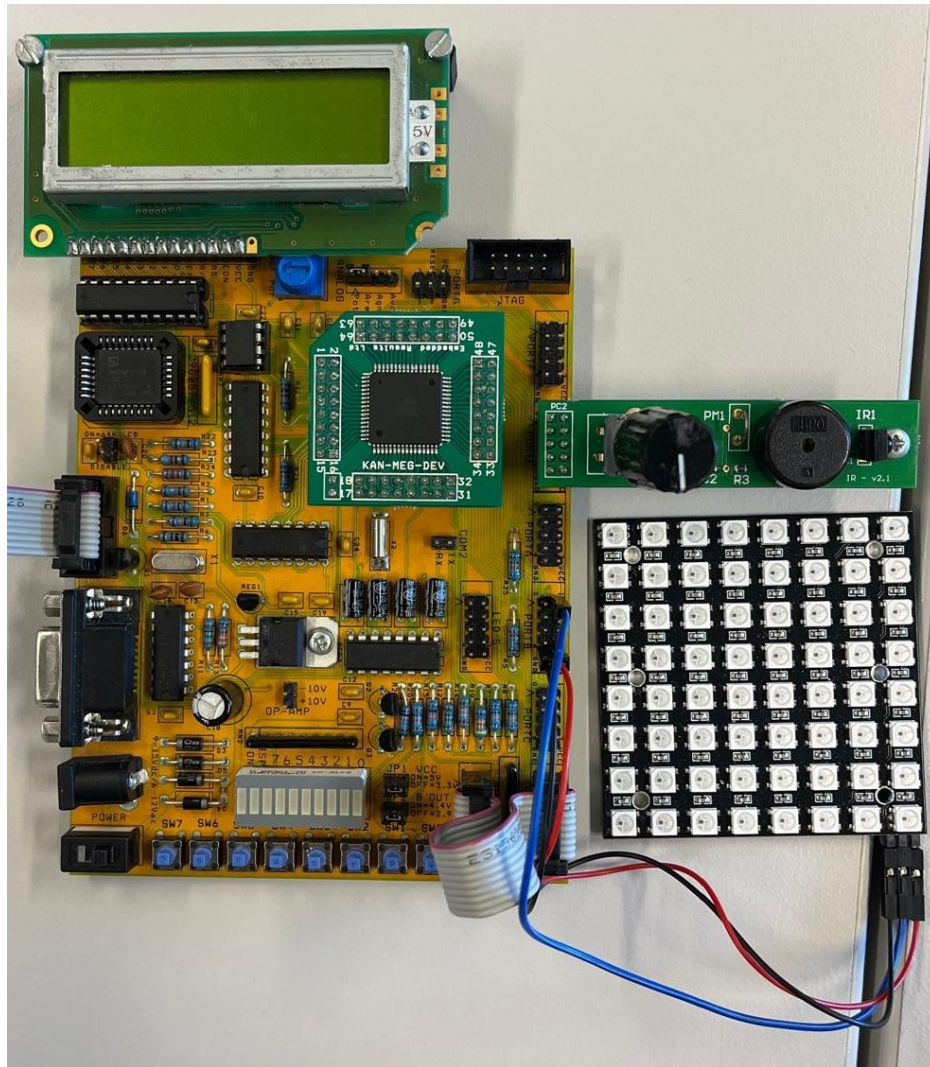


Figure 3: Connexion physique des périphériques sur la carte

5.2 Connexion software

Après la connexion matérielle, il faut connecter les bons pins aux bonnes sorties et les calibrer en entrées ou sorties.

- **Périphérique 64 LED WS2812B** : ce périphérique communique à l'aide de son câble de signal connecté sur le **PIN1** du **PORTB**. Son objectif est d'afficher les pixels définis par les fonctions de jeu dans le code. Il ne fait que recevoir des informations. On le configure donc en sortie via l'instruction : `OUTI DDRB, 0x02`, qui met le **PIN1** du **PORTB** en mode sortie. Ensuite, on envoie les informations de positionnement et de couleur en mettant à 1 ou 0 l'information avec `SBI PORTB, 1` ou `CBI PORTB, 1`. Le contrôle des LED WS2812B nécessite une précision temporelle extrême. Une **sous-routine spécifique** est utilisée pour générer le signal numérique requis. Cette routine peut s'appuyer sur un timer matériel mais ne repose pas sur une interruption classique.
- **Périphérique encodeur angulaire** : ce périphérique permet de contrôler le lancement du jeu et la position des blocs. La sortie de l'encodeur doit être en mode entrée, car on récupère les signaux envoyés sur le **PORTE**. L'instruction `OUTI DDRE, 0x00` met tous les pins du **PORTE** en mode entrée. Lors d'une rotation, les deux signaux sont décalés d'un cycle d'horloge. Le signal 1 est lancé soit plus tôt soit plus tard que le signal 2, ce qui permet de déterminer le sens de rotation. Un dernier pin gère le bouton-poussoir intégré à l'encodeur, déclenchant le début de partie.

- **Périphérique *LCD*** : ce périphérique est piloté via les sous-routines du cours, qui permettent d'écrire sur l'écran. Trois sous-routines principales sont utilisées :
 - `set_screen1` : affiche *"enter your name: "* en ligne 1 et positionne le curseur en début de ligne 2.
 - `set_screen2` : affiche *"last score :"* en ligne 1, et le score et nom précédents (récupérés en EEPROM) en ligne 2.
 - `set_screen3` : affiche *"score "* et le score courant en ligne 1, puis sauvegarde ce score en EEPROM pour la prochaine partie.



Figure 4: affichage de `set_screen1`, 2 et 3

- **Périphérique *boutons-poussoirs*** : cinq boutons sont utilisés. Le bouton 0 est connecté à l'interruption externe INT0. Lorsqu'il est pressé, il provoque un saut vers l'adresse 0x0002, qui contient un `RJMP` vers la routine d'interruption permettant de mettre le jeu en pause. Les autres boutons sont lus dans la boucle principale, via la sous-routine `writing`. Celle-ci lit l'état des boutons dans `r18` avec : `IN r18, PIND`, puis compare la valeur avec les différents cas pour permettre la saisie du nom, qui est ensuite sauvegardé en EEPROM.

A Annexe : Code Assembleur AVR

```
1 ;
2 ; ProjetApplication.asm
3 ;
4 ; Created: 26.04.2025 11:30:37
5 ; Author : GROUPE 91
6 ;
7 ;==== interrupt table ====
8 .org 0
9     jmp reset
10 .org 0x0002
11     jmp mettre_pause
12
13 ;==== interrupt service routines ====
14
15 mettre_pause: ; si pause = 1 => met 0 et inv
16
17     lds _w,pause
18     tst _w
19     breq clear
20     ldi _w,0
21     sts pause,_w
22     rjmp return
23
24
25 clear:
26     ldi _w,1
27     sts pause,_w
28
29 return:
30
31 reti
32 ;==== les includes ====
33 ;doit tre ici pour des probl mes de gestion de m moire
34 .include "definitions_memory.asm"
35 .include "macros.asm"
36 .include "definitions.asm"
37 .include "tetris.asm"
38 .include "angular_enoder.asm"
39 .include "lcd_routine.asm"
40
41
42
43 reset:
44
45     OUTI    DDRD, 0x00 ; connectbuttons to PORTC, input mode
46     LDSP    RAMEND ; Load Stack Pointer (SP)
47     OUTI    DDRE, 0x00 ; pour l'encodeur angulaire
48     OUTI    EIMSK,0b1
49     sei
50     rcall encoder_init
51     rcall set_screen1 ; affiche entre ton nom
52
53
54
55 main:
56     rcall writing ;pour crire son nom.
57
58     WAIT_MS 1; boucle rapide pour bien decoder l'encodeur
59     ldi up,0
```



```

60 rcall encoder
61 brtc skip; si on appuie sur l'encodeur on lance le jeu
62 rcall set_screen2; affiche le last score avec son nom
63
64 call start_game
65
66 rcall load_store; pour mettre jour le nom de l'ancien joueur
67
68 rcall set_screen3; affiche le score
69 WAIT_MS 3000 ; on affiche le score 3sec
70 rcall set_screen1 ; affiche entre ton nom
71
72 skip:
73
74 rjmp main

```

Listing 1: ProjetApplication.asm

```

1 /*
2  * definitions_memory.asm
3  * definition of all used memory
4  * Author: GROUPE 91
5  */
6
7 ; ***** ZONE DE DONNEES *****
8 .DSEG ; on ecrit ce qui suit dans la memoire de donnee
9
10 screen: .byte 64*3 ; cran 64 LEDs * 3 octets
11 moving_shape: .byte 64*3 ; cran 64 LEDs * 3 octets pour la piece qui tombe
12 test_collide_screen: .byte 72*3 ; ajout d'un bord
13 test_collide_moving: .byte 72*3 ; pour pouvoir simuler les déplacements
14 collision_flag: .byte 1
15 nbr_shape: .byte 1
16 random: .byte 1
17 score: .byte 1
18 pause: .byte 1 ; = 1 si pause
19 end: .byte 1 ; = 1 si perdu
20
21 .CSEG; pour pouvoir l'inclure dans un fichier de programme
22 .org 0x0045 ; apres les interrupts
23
24 ; pour debug on peut choisir la suite des pieces qui arrivent
25 liste_shape: ; seq de piece
26     .db 1, 2, 1, 3, 2, 4, 4, 3, 1, 2
27     .db 1, 3, 1, 4
28     .db 0 ; 0 pour la fin
29 str0:
30     .db "ENTER YOUR NAME:",0
31
32 str1:
33     .db "LAST SCORE: ",0
34
35 str2:
36     .db "SCORE: ",0
37
38 hextb:
39     .db "ABCDEFGHIJKLMNOPQRSTUVWXYZ",0
40
41 valeurs:
42     .db "0123456789"

```

Listing 2: definitions_memory.asm

```

1  /*
2  * definitions.asm
3  * definition des registres
4  *   Author: GROUPE 91
5  */
6
7  ; file: definitions.asm   target ATmegred28L-4MHz-STK300
8  ; purpose library, definition of addresses and constants
9  ; 20171114 A.S.
10
11 ; === definitions ===
12 .nolist           ; do not include in listing
13 .set      clock   = 4000000
14
15 .def      char     = r0      ; character (ASCII)
16 .def      _sreg    = r1      ; saves the status during interrupts
17 .def      _u       = r2      ; saves working reg u during interrupt
18 .def      u        = r3      ; scratch register (macros, routines)
19
20 .def      e0       = r4      ; temporary reg for PRINTF
21 .def      e1       = r5
22
23 .equ      c        = 8
24 .def      c0       = r8      ; 8-byte register c
25 .def      c1       = r9
26 .def      c2       = r10
27 .def      c3       = r11
28
29 .equ      d        = 12      ; 4-byte register d (overlapping with c)
30 .def      d0       = r12
31 .def      d1       = r13
32 .def      d2       = r14
33 .def      d3       = r15
34
35 .def      w        = r16      ; working register for macros
36 .def      _w       = r17      ; working register for interrupts
37
38
39 ;registres pour le panneau led
40
41 .def      green    = r18
42 .def      red      = r19
43 .def      blue     = r20
44
45 ;registres pour les loops
46
47 .def      loop      = r21
48 .def      loop2     = r22
49 .def      main_loop_count = r23
50
51 ;registres pour l'encodeur
52
53 .def      up        = r24
54 .def      down      = r25
55
56 .equ      px        = 26      ; pointer x
57 .equ      py        = 28      ; pointer y
58 .equ      pz        = 30      ; pointer z

```

Listing 3: definitions.asm

```

1  /*
2  * memory_utils.asm
3  * All macros and subroutines that modify memory are defined below.
4  *   Author: GROUPE 91
5  */
6
7  ; Macro pour initialiser un bloc mmoire une valeur sp cifi e , valeur de z
   modifi e
8  ; Usage: INIT_MEMORY(adresse_debut, taille, valeur)
9  .macro INIT_MEMORY
10     ldi loop, @1
11     ldi r17, @2
12     ldi z1, low(@0)
13     ldi zh, high(@0)
14
15 init_loop:
16     st Z+, r17
17     dec loop
18     brne init_loop
19
20 .endm
21
22 ; Macro pour load un pixel en mmoire une adresse (z), valeur de z modifi e de 3
23 ; Usage: LOAD_RGB(Green, Red, Blue)
24 .macro LOAD_RGB
25
26     ldi green, @0
27     st Z+, green
28     ldi red, @1
29     st Z+, red
30     ldi blue, @2
31     st Z+, blue
32
33 .endm
34
35 ; Macro pour copier un bloc mmoire un endroit sp cifi , valeur de z modifi e
36 ; Usage: COPY_MEMORY(adresse_du_copi , adresse_du_coll , nbr_pix)
37 .macro COPY_MEMORY
38
39     ldi z1, low(@1)
40     ldi zh, high(@1)
41
42     ldi y1, low(@0)
43     ldi yh, high(@0)
44
45     ldi loop, @2
46     copy_loop:
47
48         rcall copy_rgb
49         dec loop
50         brne copy_loop
51
52 .endm
53
54 ; Macro pour add deux bloc mmoire
55 ; Usage: ADD_MEMORY(adresse_du_dest, adresse_du_add)
56 .macro ADD_MEMORY
57     ; Initialiser pointeurs
58     ldi z1, low(@0)
59     ldi zh, high(@0)
60

```

```

61     ldi yl, low(@1)
62     ldi yh, high(@1)
63
64     ldi loop, 64*3
65
66 loop_add:
67     ld  r16, Z+
68     ld  r17, Y+
69     add r16, r17
70     sbiw z1, 1
71     st  z+, r16
72
73     dec loop
74     brne loop_add
75
76 .endm
77
78 ; Macro pour shift    droit un carr  m moire de 8*9 pixel (1 pixel = 3 octet)
79 ; valeur de z,y modifi e
80 ; Usage: MEMORY_SHIFT_RIGHT(adresse_du_right_shift)
81 .macro MEMORY_SHIFT_RIGHT
82
83     ldi z1, low(@0 + 216)
84     ldi zh, high(@0 + 216)
85
86     ldi yl, low(@0 + 213)
87     ldi yh, high(@0 + 213)
88
89     ldi loop2, 9 ; 9 lignes
90     memory_shift_right_loop:
91         ldi loop, 21; copie les 3*7 octet des 7 pixel
92         octet_loop:
93             ld r16, -Y
94             st -Z, r16
95
96         dec loop
97         brne octet_loop
98
99         ldi loop, 3; met    0 le nouveau pixel
100        zero_loop:
101            ld r16, -Y ;juste pour dec y
102            ldi r16, 0x00
103            st -Z, r16
104
105        dec loop
106        brne zero_loop
107    dec loop2
108    brne memory_shift_right_loop
109
110 .endm
111
112 ; Macro pour shift    droit un carr  m moire de 8*9 pixel (1 pixel = 3 octet)
113 ; valeur de z,y modifi e
114 ; Usage: MEMORY_SHIFT_LEFT(adresse_du_left_shift)
115 .macro MEMORY_SHIFT_LEFT
116
117     ldi z1, low(@0)
118     ldi zh, high(@0)
119
120     ldi yl, low(@0 + 3)
121     ldi yh, high(@0 + 3)
122

```

```

123     ldi loop2, 9 ; 9 lignes
124     memory_shift_left_loop:
125         ldi loop, 21; copie les 3*7 octet des 7 pixel
126         octet_loop:
127             ld r16, Y+
128             st Z+, r16
129
130         dec loop
131         brne octet_loop
132
133         ldi loop, 3; met 0 le nouveau pixel
134         zero_loop:
135             ld r16, Y+ ;juste pour inc y
136             ldi r16, 0x00
137             st Z+, r16
138
139         dec loop
140         brne zero_loop
141     dec loop2
142     brne memory_shift_left_loop
143
144 .endm
145 ; Macro pour decendre un carr m moire de 8*9 pixel (1 pixel = 3 octet) de une ligne
146 ; Usage: MEMORY_SHIFT_DOWN(adresse_du_down_shift,nbr de lingnes(registre))
147 .macro MEMORY_SHIFT_DOWN
148
149     mov c0,loop
150     mov w, @1
151     ldi loop,24
152     mul loop, w
153     mov loop,r0
154     ;ldi loop, 216
155     mov r24, loop
156
157
158     ldi x1, low(@0)
159     ldi xh, high(@0)
160     ADDX loop; met l'adresse max
161
162     subi loop, 24
163     mov r23, loop
164     ;ADDI loop, 24
165
166     ldi y1, low(@0)
167     ldi yh, high(@0)
168     ADDY loop; met l'adresse de 'avant derni re ligne
169
170 shift_down_loop:
171     ld r16, -Y
172     st -x, r16
173
174     dec loop
175     brne shift_down_loop
176
177     ; Mettre les 24 premiers octets z ro
178     ldi loop, 24
179     ldi x1, low(@0)
180     ldi xh, high(@0)
181     ldi r16, 0
182
183 zero_fill:
184     st x+, r16

```

```

185     dec loop
186     brne zero_fill
187
188     mov loop, c0
189
190 .endm
191
192
193 ; Macro pour charger un carr (2*2)pixel dans les deux lignes de 8 pixel l'adresse
194 ; donn
195 ; Usage: MEMORY_ADD_SQUARE(adresse_du_carr )
196 .macro MEMORY_ADD_SQUARE
197
198     INIT_MEMORY @0, 8*2*3, 0x00
199
200     ; LED 4 et 5 de la 1 re ligne
201     ldi z1, low(@0 + 9)
202     ldi zh, high(@0 + 9)
203
204     ldi loop, 2
205     call loop_square
206
207     ; LED 4 et 5 de la 2 me ligne
208     ldi z1, low(@0 + 33)
209     ldi zh, high(@0 + 33)
210
211     ldi loop, 2
212     call loop_square; call car la macro peut tre tr s loin
213 .endm
214
215 ;boucle pour mettre une barre de 2 du carr
216 loop_square:
217     LOAD_RGB 0x01, 0x0F, 0x01
218     dec loop
219     brne loop_square
220
221 ret
222
223 ; Macro pour charger un Z horizontal (2*2)pixel dans les deux lignes de 8 pixel l'
224 ; adresse donn
225 ; Usage: MEMORY_ADD_Z_H(adresse_du_z)
226 .macro MEMORY_ADD_Z_H
227
228     INIT_MEMORY @0, 8*2*3, 0x00
229
230     ; LED 4 et 5 de la 1 re ligne
231     ldi z1, low(@0 + 9)
232     ldi zh, high(@0 + 9)
233
234     ldi loop, 2
235     call loop_z_h
236
237     ; LED 3 et 4 de la 2 me ligne
238     ldi z1, low(@0 + 33)
239     ldi zh, high(@0 + 33)
240
241     ldi loop, 1
242     call loop_z_h
243 .endm
244
245 ;boucle pour mettre une barre de 2 du carr
246 loop_z_h:
247     LOAD_RGB 0x0F, 0x01, 0x01
248     dec loop

```



```

245         brne loop_z_h
246     ret
247
248     ; Macro pour charger une barre verticale dans les trois lignes de 8 pixel    l'adresse
249     ; donn
250     ; Usage: MEMORY_ADD_BARRE(adresse_du_z)
251     .macro MEMORY_ADD_BARRE
252
253         INIT_MEMORY @0, 8*3*3, 0x00
254
255         ; LED 4 de la 1 re ligne
256         ldi z1, low(@0 + 9)
257         ldi zh, high(@0 + 9)
258
259         LOAD_RGB 0x01, 0x0F, 0x0F
260
261         ; LED 4 de la 2 me ligne
262         ldi z1, low(@0 + 33)
263         ldi zh, high(@0 + 33)
264         LOAD_RGB 0x01, 0x0F, 0x0F
265
266         ; LED 4 de la 3 me ligne
267         ldi z1, low(@0 + 57)
268         ldi zh, high(@0 + 57)
269         LOAD_RGB 0x01, 0x0F, 0x0F
270
271     .endm
272
273     .macro MEMORY_ADD_T
274
275         INIT_MEMORY @0, 8*2*3, 0x00
276
277         ; LED 4, 5 et 6 de la 1 re ligne
278         ldi z1, low(@0 + 9)
279         ldi zh, high(@0 + 9)
280
281         ldi loop, 3
282         call loop_t
283
284         ; LED 5 de la 2 me ligne
285         ldi z1, low(@0 + 36)
286         ldi zh, high(@0 + 36)
287         LOAD_RGB 0x01, 0x01, 0x0F
288
289     .endm
290
291     loop_t:
292         LOAD_RGB 0x01, 0x01, 0x0F
293         dec loop
294         brne loop_t
295     ret
296
297     ; Sous routine pour copier le RGB pointer par y    l'endroit point par z
298     copy_rgb:
299         ld green, y+
300         ld red, y+
301         ld blue, y+
302         st z+, green
303         st z+, red
304         st z+, blue
305

```

```

306 ret
307
308 ; Macro pour set un flag en memoire data
309 ; Usage: SET_COLL_FLAG(adresse_du_flag)
310 .macro SET_FLAG
311     ldi r16, 1
312     sts @0, r16
313 .endm
314
315 ; Macro pour clear un flag en memoire data
316 ; Usage: CLR_FLAG(adresse_du_flag)
317 .macro CLR_FLAG
318     ldi r16, 0
319     sts @0, r16
320 .endm

```

Listing 4: memory_utils.asm

```

1  /*
2   * tetris.asm
3   * le code principale du jeu
4   * Author: GROUPE 91
5   */
6  .include "led_64.asm" ;le perif utilis
7  .include "random.asm"
8  .include "memory_utils"
9
10 start_game:
11
12     ;met tout les params de la data seg    0
13     rcall  led_64_init
14     rcall  init_screen
15     rcall  init_moving_shape
16     rcall  init_test_collide_moving
17     rcall  init_test_collide_screen
18     CLR_FLAG nbr_shape
19     CLR_FLAG score
20     CLR_FLAG end
21     CLR_FLAG pause
22
23     rcall  load_new_piece
24
25 main_game:
26     rcall display_screen_and_moving
27
28     ; pour debug avec les boutons:
29     /*in r17, PIND
30
31     sbrc r17, 3
32     rjmp button_right_not_pressed
33     rcall move_right
34     rcall display_screen_and_moving
35
36     button_right_not_pressed:
37
38     sbrc r17, 4
39     rjmp button_left_not_pressed
40     rcall move_left
41     rcall display_screen_and_moving
42
43     button_left_not_pressed:*/
44

```

```

45 ldi main_loop_count, 255
46 main_game_loop:
47     ldi up,0
48     ldi down,0
49     rcall encoder
50     tst up
51     breq no_move
52
53     brpl right
54     brmi left
55
56
57
58 left:
59     rcall move_left
60     rcall display_screen_and_moving
61     rjmp no_move
62
63 right:
64     rcall move_right
65     rcall display_screen_and_moving
66 no_move:
67     WAIT_MS 2
68     dec main_loop_count
69     brne main_game_loop
70
71     rcall move_down
72     rcall display_screen_and_moving
73
74     rcall clear_line
75     rcall display_screen_and_moving
76
77     rcall clear_line; 2 appellees car il peut y avoir 2 lignes a clear
78     rcall display_screen_and_moving
79
80     pause_loop;; si le jeu est en pause on attend une deuxieme interruption pour
81         repartir
82     lds r16, pause
83     tst r16
84     brne pause_loop
85
86     WAIT_MS 10
87
88     lds r16, end ; si end = 1 on termine le jeu
89     tst r16
90     breq continue
91
92     ret
93 continue:
94
95     jmp main_game
96
97 ;==== AFFICHAGE COMPLET ====
98 ; affiche le screen et le moving shape
99 ;utilise registre : green, red, bleu, r13,r14,r15
100 display_screen_and_moving:
101     ; Initialiser pointeurs
102     ldi z1, low(screen)
103     ldi zh, high(screen)
104
105     ldi y1, low(moving_shape)

```

```

106     ldi yh, high(moving_shape)
107
108     ldi loop, 64
109
110 display_loop:
111     ; Lire pixel de screen
112     ld green, Z+
113     ld red, Z+
114     ld blue, Z+
115
116     ; Lire pixel de moving_shape
117     ld r13, Y+
118     ld r14, Y+
119     ld r15, Y+
120
121     ; Additionner screen + moving
122     add green, r13
123     add red, r14
124     add blue, r15
125
126     rcall led_64_write
127
128     dec loop
129     brne display_loop
130
131     ; Quand fini, reset des LEDs
132     rcall led_64_reset
133
134     ret
135 ;==== MOVE LEFT ====
136 ; bouge la moving shape    gauche si pas de collision si collision retourne
137 ; utilise registre : 16
138 move_left:
139
140     rcall load_test_collide_moving
141     rcall load_test_collide_screen
142
143     MEMORY_SHIFT_LEFT test_collide_moving
144
145     rcall collision ; pour la collision avec les autres pi ces
146
147     ; on verifie apres chaque test
148     lds r16, collision_flag
149     tst r16
150     brne collision_left ; si != 0 saute collision_left
151
152     rcall test_side_collide_left ; pour la collision avec le bord
153
154     ; on verifie apres chaque test
155     lds r16, collision_flag
156     tst r16
157     brne collision_left ; si != 0 saute collision_left
158
159     MEMORY_SHIFT_LEFT moving_shape ; pas de collision on bouge
160
161     collision_left: ; si il y a une collision on annule le déplacement
162
163     ret
164
165     ; pour tester les collisions    gauche appeler dans move_left
166     ; utilise r16 r24 et r1 (tjrs 0)
167 test_side_collide_left: ; pour pouvoir aller    gauche la ligne de gauche doit tre

```

```

168     vide
169     ldi z1, low(moving_shape)
170     ldi zh, high(moving_shape)
171
172     ; R initialiser collision_flag      0
173     CLR_FLAG collision_flag
174
175     ldi loop, 8 ; 8 lignes
176
177     collision_left_loop:
178     ld r16, Z
179     tst r16
180     breq PC+5 ; car la macro SET_FLAG      2 lignes
181     SET_FLAG collision_flag
182     ret
183
184     ; sinon on avance Z de 24 octets (pour lire la ligne suivante)
185     ldi r24, 24
186     clr r1
187     add z1, r24
188     adc zh, r1 ; si besoin ajoute la carry (r1=0)
189     dec loop
190     brne collision_left_loop
191     ret
192
193
194     ;==== MOVE RIGHT ====
195     ; bouge la moving shape      droite si pas de collision si collision retourne
196     ; utilise registre : 16
197     ; sous routine tr s proche de MOVE LEFT
198     move_right:
199
200     rcall load_test_collide_moving
201     rcall load_test_collide_screen
202
203     MEMORY_SHIFT_RIGHT test_collide_moving
204
205     rcall collision ; pour la collision avec les autres pi ces
206
207     ; on verifie apres chaque test
208     lds r16, collision_flag
209     tst r16
210     brne collision_right; si != 0 saut      collision_right
211
212     rcall test_side_collide_right ; pour la collision avec le bord
213
214     ; on verifie apres chaque test
215     lds r16, collision_flag
216     tst r16
217     brne collision_right; si != 0 saut      collision_right
218
219     MEMORY_SHIFT_RIGHT moving_shape; pas de collision on bouge
220
221     collision_right: ; si il y a une collision on annule de déplacement
222
223     ret
224
225     ; pour tester les collisions      droite appeler dans move_right
226     ; utilise r16 r24 et r1 (tjrs      0)
227     test_side_collide_right:
228     ldi z1, low(moving_shape + 21) ; cette fois ci on regarde de l'autre cot

```

```

229     ldi zh, high(moving_shape + 21)
230
231     ; R initialiser collision_flag      0
232     CLR_FLAG collision_flag
233
234     ldi loop, 8 ; 8 lignes
235
236     collision_right_loop:
237     ld r16, Z
238     tst r16
239     breq PC +5 ; car la macro SET_FLAG      2 lignes
240     SET_FLAG collision_flag
241     ret
242
243     ; sinon on avance Z de 24 octets (pour lire la ligne suivante)
244     ldi r24, 24
245     clr r1
246     add z1, r24
247     adc zh, r1 ; si besoin ajoute la carry (r1=0)
248     dec loop
249     brne collision_right_loop
250     ret
251
252
253     ;==== MOVE DOWN ====
254     ; bouge la moving shape vers le bas si pas de collision
255     ; si collision ajoute dans screen la moving shape
256     ; utilise registre : 16
257     move_down:
258
259     rcall load_test_collide_moving
260     rcall load_test_collide_screen
261
262     ldi r16, 9
263     MEMORY_SHIFT_DOWN test_collide_moving, r16
264
265     rcall collision
266
267     ; Lire la valeur du flag
268     lds r16, collision_flag
269     tst r16
270     breq no_down_collision; si = 0 saut      no_collisoion
271
272     ; Si collision_flag = 1 :
273     rcall handle_down_collision
274     rjmp fin
275
276     no_down_collision:
277     ldi r16, 9
278     MEMORY_SHIFT_DOWN moving_shape, r16
279
280     fin:
281
282     ret
283
284     ;==== COLLISION ====
285     ; test les collisions entre test_collide_screen et test_collide_moving
286     ; et met      jours collision_flag
287     ; utilise registre : 18,19
288     collision:
289
290     ; Initialisation des pointeurs Z et Y

```

```

291     ldi z1, low(test_collide_screen)
292     ldi zh, high(test_collide_screen)
293
294     ldi y1, low(test_collide_moving)
295     ldi yh, high(test_collide_moving)
296
297     ; R initialiser collision_flag      0
298     CLR_FLAG collision_flag
299
300     ldi loop, 72*3          ; nombre d'octets      tester
301
302     collision_loop:
303         ld r18, Z+
304         ld r19, Y+
305
306         tst r18
307         breq skip_flag
308
309         tst r19
310         breq skip_flag
311
312         SET_FLAG collision_flag
313         ret
314
315     skip_flag:
316         dec loop
317         brne collision_loop
318
319     ret
320
321     ;==== HANDLE DOWN COLLISION ====
322     ; lors d'une collision on ajoute la moving shape dans le screen et load une nouvelle
323     ; forme dans moving_shape
324     ; si collision fin de jeu
325     ;utilise registre : 16
326     handle_down_collision:
327         rcall add_to_screen
328
329         rcall init_moving_shape
330         rcall load_new_piece
331
332         rcall load_test_collide_moving
333         rcall load_test_collide_screen
334
335         rcall collision
336
337         lds r16, collision_flag
338         tst r16
339         breq no_collision
340
341         rcall display_screen_and_moving
342         rcall game_over
343
344     no_collision:
345     ret
346     ;==== GAME OVER ====
347     ;fin de jeu
348     ;utilise registre : 16
349     game_over:
350         rcall add_to_screen
351         WAIT_MS 450

```

```

352
353     rcall init_moving_shape
354     rcall init_screen
355     ldi r16, 1
356     sts end,r16
357
358
359 ret
360 ;==== CLEAR LINE ====
361 ;vide les lignes qui sont pleine
362 ;utilise registre : 18,8
363 clear_line:
364
365     ldi z1, low(screen + 192)
366     ldi zh, high(screen + 192)
367
368
369     ldi loop, 8
370
371     check_line_loop:
372
373         ldi loop2,8*3
374         check_pix_loop:
375             ld r18, -Z
376
377             tst r18
378             breq skip_line
379             dec loop2
380             brne check_pix_loop
381
382         MEMORY_SHIFT_DOWN screen, loop ; ligne pleine on shift tt vers le bas et crase
383             la ligne
384         lds r8, score
385         inc r8
386         sts score,r8
387 ; si on voit un 0 on peu skipe le reste de la ligne
388     skip_line:
389         subi loop2, 1
390         sub z1,loop2
391         clr r1
392         sbc zh, r1
393         dec loop
394         brne check_line_loop
395
396
397 ret
398
399 ;==== LOAD NEW PIECE ====
400 ;charge la pi ce suivante dans moving shape
401 ;utilise registre : 1,22,20,15
402 load_new_piece:
403
404     ;pour forcer manueleemnet des pi ces bien pour debug
405     /*; Charger l'index
406     lds r15, nbr_shape
407
408
409     ldi z1, low(liste_shape * 2)
410     ldi zh, high(liste_shape * 2)
411     clr r1
412     add z1, r15

```



```

413     adc zh, r1
414
415     ;on lit la liste
416     lpm r22, Z*/
417     ; pour le random
418     rcall random_1_4
419     mov r22, r1
420
421     ; Appeler la forme correspondante
422     cpi r22, 1
423     breq load_1
424     cpi r22, 2
425     breq load_2
426     cpi r22, 3
427     breq load_3
428     cpi r22, 4
429     breq load_4
430     .
431     .; si plus de formes
432     .
433     rjmp continue_inc
434
435 load_1:
436     rcall load_square
437     rjmp continue_inc
438 load_2:
439     rcall load_z_h
440     rjmp continue_inc
441 load_3:
442     rcall load_t
443     rjmp continue_inc
444 load_4:
445     rcall load_barre
446     rjmp continue_inc
447 ; ...
448
449 continue_inc:
450     inc r15
451     sts nbr_shape, r15
452
453     ;si on est arriver au bout de la liste de forme (=0) on se remet au d but
454     ldi ZL, low(liste_shape * 2)
455     ldi ZH, high(liste_shape * 2)
456     add ZL, r15
457     adc ZH, r1
458     lpm r20, Z;prochaine valeur
459
460     tst r20
461     brne end_piece
462
463     ; Sinon on remet l index      0
464     ldi r21, 0
465     sts nbr_shape, r21
466
467 end_piece:
468     ret
469
470 ;===== MANIPULATION MEMOIRE =====
471 ;les sous routines suivantes utilise les macro de memory_utils.asm
472 ;fais de cette mani re pour avoir de 'abstraction sur l'acc s m moire
473
474 ;==== ADD TO SCREEN ====

```

```

475
476 add_to_screen:
477     ADD_MEMORY    screen, moving_shape
478 ret
479
480 ;==== INIT SCREEN ====
481
482 init_screen:
483     INIT_MEMORY   screen, 64*3, 0x00
484 ret
485
486 ;==== INIT MOVING SHAPE ====
487
488 init_moving_shape:
489     INIT_MEMORY   moving_shape, 64*3, 0x00
490 ret
491
492 ;==== INIT TEST COLLIDE MOVING ====
493
494 init_test_collide_moving:
495     INIT_MEMORY   test_collide_moving, 72*3, 0x00
496 ret
497
498 ;==== INIT TEST COLLIDE SCREEN ====
499
500 init_test_collide_screen:
501     INIT_MEMORY   test_collide_screen, 72*3, 0x00
502 ret
503
504 ;==== LOAD SQUAR ====
505
506 load_square:
507     INIT_MEMORY   moving_shape, 64*3, 0x00
508     MEMORY_ADD_SQUARE moving_shape
509 ret
510 ;==== LOAD Z HORIZONTAL ====
511 load_z_h:
512     INIT_MEMORY   moving_shape, 64*3, 0x00
513     MEMORY_ADD_Z_H moving_shape
514 ret
515 ;==== LOAD T HORIZONTAL ====
516 load_t:
517     INIT_MEMORY   moving_shape, 64*3, 0x00
518     MEMORY_ADD_T moving_shape
519
520 ret
521 ;==== LOAD BARRE VERTICAL ====
522 load_barre:
523     INIT_MEMORY   moving_shape, 64*3, 0x00
524     MEMORY_ADD_BARRE moving_shape
525
526 ret
527 ;==== LOAD TEST COLLIDE MOVING ====
528 load_test_collide_moving:
529     INIT_MEMORY   test_collide_moving, 72*3, 0x00
530     COPY_MEMORY   moving_shape, test_collide_moving, 64
531
532 ret
533 ;==== LOAD TEST COLLIDE SCREEN ====
534 load_test_collide_screen:
535
536     INIT_MEMORY   test_collide_screen, 72*3, 0x00

```

```

537 COPY_MEMORY screen, test_collide_screen, 64
538
539 ; la zone m moire test_collide a une bordure (derniere ligne) 0x0F, 0x0F, 0x0F
540 INIT_MEMORY test_collide_screen + 64*3, 8*3, 0x0F
541
542 ret

```

Listing 5: tetris.asm

```

1  /*
2  * Random.asm
3  * pour le random du jeu
4  * Author: GROUPE 91
5  */
6  init_random:
7      ; TODO: use EEPROM for seed
8      ldi w, 0x00
9      sts random, w
10 ret
11
12 ; uses r16, r0, r1
13 ; Linear congruential generator
14 random_byte:
15     lds r0, random
16     ldi w, 45 ; 4* un nnr premier (11) + 1
17     mul r0, w
18     ldi w, 213; doit tre impaire
19     add r0, w
20     sts random, r0
21 ret
22
23 ; on trouve dans r1 une val random 1 and 4
24 random_1_4:
25     rcall random_byte ; r0 un random byte
26     ldi w, 4
27     mul r0, w
28     inc r1
29 ret

```

Listing 6: Random.asm

```

1  /*
2  * lcd_routine.asm
3  * toutes les sous routines utiles pour le LCD
4  * Author: GROUPE 91
5  */
6
7  ;==== SET SCREEN 1 ====
8  ;affiche "ENTER YOUR NAME:" en ligne 1 et place le curseur en ligne 2
9  ;utilise registre : 18,19,20,x,z
10
11 .include "eeprom.asm"
12 .include "lcd.asm"
13
14 set_screen1:
15     rcall LCD_init
16
17     ldi x1, 0x10
18     ldi xh, 0x00
19     ldi z1, low(2*str0) ; load pointer to string
20     ldi zh, high(2*str0)
21     rcall LCD_putstr

```

```

22     ldi r19, 0x40
23     mov r18,r19
24     rcall LCD_pos
25     ldi r20, 0
26     rcall ECRITURE
27     mov r18, r19
28     rcall LCD_pos
29     ret
30
31 ;==== SET SCREEN 2 ====
32 ; affiche "last score:" et le last score avec son nom
33 ;utilise registre : 18,19,25,x,z
34 set_screen2:
35     rcall LCD_clear
36     ldi z1, low(2*str1) ; load pointer to string
37     ldi zh, high(2*str1)
38     rcall LCD_putstring
39     ldi r19, 0x40
40     mov r18,r19
41     rcall LCD_pos
42     ldi r25, 5
43     ldi x1, 0x00
44     ldi xh, 0x00
45 loop_name:
46     rcall eeprom_load
47     rcall LCD_put1
48     dec r25
49     brne loop_name
50     rcall eeprom_load
51     rcall putdec
52     ret
53
54 ;==== SET SCREEN 3 ====
55 ; affiche "score:" et le score de la partie
56 ;utilise registre : 18,19,x,z
57 set_screen3:
58     rcall LCD_clear
59     ldi z1, low(2*str2) ; load pointer to string
60     ldi zh, high(2*str2)
61     rcall LCD_putstring
62     ldi r19, 0x40
63     mov r18,r19
64     rcall LCD_pos
65     lds r18, score
66     ldi x1, 0x05
67     ldi xh, 0x00
68     rcall eeprom_store
69     rcall putdec
70     ret
71
72 ;==== writing ====
73 ; lis les boutons-poussoirs et renvoie vers leur sous-routines
74 ;utilise registre : 18,23
75 writing:
76     in r18,PIND ; read switches
77     ldi r23,0b11111101
78     cp r18, r23
79     brne PC+2
80     rcall plus_letter
81     ldi r23,0b11111011
82     cp r18, r23
83     brne PC+2

```

```

84     rcall moins_letter
85     ldi r23,0b11110111
86     cp r18, r23
87     brne PC+2
88     rcall LCD_put0
89     ldi r23,0b11110111
90     cp r18, r23
91     brne PC+2
92     rcall reset_name
93     wait_MS 200
94     ret
95
96 ;==== plus_letter ====
97 ; inc r mente les lettre lors de l' critue du nom
98 ;utilise registre : 18,19, r20
99 plus_letter:
100     ADDI r20, 1
101     rcall ECRITURE
102     mov r18, r19
103     rcall LCD_pos
104     ret
105
106 ;==== moins_letter ====
107 ; d cr mente les lettre lors de l' critue du nom
108 ;utilise registre : 18,19, r20
109 moins_letter:
110     subi r20, 1
111     rcall ECRITURE
112     mov r18, r19
113     rcall LCD_pos
114     ret
115
116 ;==== reset_name ====
117 ; efface le nom et replace le curseur au d but de la ligne
118 reset_name:
119     rcall LCD_clear
120     rcall set_screen1
121     ret
122
123 ;==== LCD_put0 ====
124 ; cris le nom et le store dans la EEPROM
125 ;utilise registre : 18,19, r20, x
126 LCD_put0:
127     rcall ECRITURE
128     inc r19
129     mov r18, r19
130     rcall LCD_pos
131     mov r18, r20
132     rcall eeprom_store
133     inc x1
134     ldi r20, 0
135     rcall ECRITURE
136     mov r18, r19
137     rcall LCD_pos
138     ret
139
140 ;==== LCD_put1 ====
141 ; cris le last name
142 ;utilise registre : 18,19, r20, x
143 LCD_put1:
144     mov r20, r18
145     rcall ECRITURE

```

```

146     inc r19
147     mov r18, r19
148     rcall LCD_pos
149     inc x1
150     ret
151
152     ;==== ECRITURE ====
153     ;   cris   une lettre
154     ;utilise registre : r0, r1, r18, r20, z
155 ECRITURE:
156     ldi z1, low(2*hextb)
157     ldi zh, high(2*hextb)
158     add z1, r20
159     clr r1
160     adc zh, r1
161     lpm
162     mov r18, r0
163     rcall LCD_putc
164     ret
165
166     ;==== LCD_putstring ====
167     ;   cris   un texte
168     ;utilise registre : r0, r18, z
169 LCD_putstring:
170     ; in      z
171     lpm                ; load program memory into r0
172     tst  r0            ; test for end of string
173     breq done
174     mov  r18, r0        ; load argument
175     rcall LCD_putc
176     adiw z1,1          ; increase pointer address
177     rjmp LCD_putstring ; restart until end of string
178 done:    ret

```

Listing 7: lcd_routine.asm